

MODULE 4 – EMERGING DATABASE MODELS, TECHNOLOGIES AND APPLICATIONS

UNIT 1: RELATIONAL AND NON-RELATIONAL DATABASES

1. Introduction

For most of the last 40 years, businesses relied on relational database management systems (RDBMSs)—that used Structured Query Language (SQL) as the programming language. ScaleGrid reports that 60.5% of the commonly used databases are SQL-based RDBMS. Applications in domains such as Multimedia, Geographical Information Systems, and digital libraries demand a completely different set of requirements in terms of the underlying database models. The conventional relational database model is no longer appropriate for these types of data. Furthermore the volume of data is typically significantly larger than in classical database systems. In addition, indexing, retrieving and analyzing data types require specialized functionality not

available in conventional database systems. The unit will cover the some discussions and the requirements and description of emerging databases.

1. Objectives:

At the end of this unit, you should be able to:

1. understand the differences between relational and non-relational databases
2. understand the various database models and their limitations
3. describe the various emerging databases

3.2 SQL-based Database Management Systems:

Relational database management systems (RDBMSs) use SQL, a database management language that offers a highly organized and structured approach to information management. Similar to the way a phone book has different categories of information (name, number, address, etc.) for each line of data, relational databases apply strict, categorical parameters that allow database users to easily organize, access, and maintain information within those parameters. The primary reasons why SQL-based RDBMSs continue to dominate are because they are highly stable and reliable, adhere to a standard that integrates seamlessly with popular software stacks and have been used for more than 40 years. Popular examples of SQL database engines include:

1. Oracle Database
2. MySQL
3. Microsoft SQL Server

3.3 Advantages of RDBMS:

1. **ACID compliance:** If a database system is "ACID compliant," it satisfies a set of priorities that measure the atomicity, consistency, isolation, and durability of database systems. The more ACID-compliant a database is, the more it serves to guarantee the validity of database transactions, reduce anomalies, safeguard data integrity, and create stable database systems. Generally, SQL-based RDBMSs achieve a high level of ACID compliance, but NoSQL databases give up this distinction to gain speed and flexibility when dealing with unstructured data.

2. Ideal for consistent data systems: With an SQL-based RDBMS, information will remain in its original structure. They offer great speed and stability especially if it does not involve massive amounts of data.
3. Better support options: Because RDBMS databases have been around for over 40 years, it's easier to get support, add-on products, and integrate data from other systems.

1. *Disadvantages of RDBMS:*

1. **Scalability challenges and difficulties with sharing:** RDBMSs have a more difficult time scaling up in response to massive growth compared to NoSQL databases. These databases also present challenges when it comes to sharing. On the other hand, a non-relational database system (NoSQL-based) handle scaling and growth better than relational databases.
2. Less efficient with NoSQL formats: Most RDBMSs are now compatible with NoSQL data formats, but they don't work with them as efficiently as non-relational databases.
3. Another characteristic of conventional databases is that there are hardly international standards available or used for the content of the databases, being the data that is entered by its users. This typically means that local conventions are applied to limit the diversity of data that may be entered in those databases. As local conventions usually differ from other local conventions this has as disadvantage that data that are entered in one database cannot be compared or integrated with data in other databases, even if those database structures are the same and even if the application domain of the databases is the same.

3.5 Non-Relational Database Systems (NoSQL-based)

When tasked with managing large amounts of unstructured data—like text from emails and customer surveys, data collected by a network of mobile apps, or random social media information. The information is disorganized. There is no clearly-defined schema like you would find an RDBMS. You cannot store it in an RDBMS. But you can with a non relational (or NoSQL) database system. Another reason why non-

relational databases are important is that they work with NoSQL formats like JSON, which has become essential for web-based applications that let websites update "live" without needing to refresh the page.

3.5.1 Advantages of Non-relational database systems:

1. **Excellent for handling "big data" analytics:** The main reason why NoSQL databases are becoming more popular is that they remove the bottleneck of needing to categorize and apply strict structures to massive amounts of information. NoSQL databases like HBase, Cassandra, and CouchDB support the speed and efficiency of server operations while offering the capacity to work with large amounts of data.
2. **No limits on types of data you can store:** NoSQL databases give you unlimited freedom to store diverse types of data in the same place. This offers the flexibility to add new and different types of data to your database at any time.
3. **Easier to scale:** NoSQL databases are easier to scale. They are designed to be fragmented across multiple data centers without much difficulty.
4. **No data preparation required:** When there isn't time to design a complex model, and you need to get a database running fast, non-relational databases save a lot of time.

3.5.2 Disadvantages of Non-Relational database systems:

1. **More difficult to find support:** Because the NoSQL community doesn't have 40 years of history and development behind it, it could be more difficult to find experienced users when you in need support.
2. **Lack of tools:** Another disadvantage relating to newness is that—compared to SQL-based RDBMS solutions—there aren't as many tools to assist with performance testing and analysis.
3. **Compatibility and standardization challenges:** Newer NoSQL database systems also lack the high degree of compatibility and standardization offered by SQL-based alternatives. Therefore, you may find that the data in your non-

relational database management system doesn't readily integrate with other products and services.

1. *Types of NoSQL Database engines:*

1. Key-value stores, such as Redis and Amazon DynamoDB, are extremely simple database management systems that store only key-value pairs and provide basic functionality for retrieving the value associated with a known key. The simplicity of key-value stores makes these database management systems particularly well-suited to embedded databases, where the stored data is not particularly complex and speed is of paramount importance.
2. Wide column stores, such as Cassandra, Scylla, and HBase, are schema-agnostic systems that enable users to store data in column families or tables, a single row of which can be thought of as a record — a multi-dimensional key-value store. These solutions are designed with the goal of scaling well enough to manage petabytes of data across as many as thousands of commodity servers in a massive, distributed system. Although technically schema-free, wide column stores like Scylla and Cassandra use an SQL variant called CQL for data definition and manipulation, making them straightforward to those already familiar with RDBMS.
3. Document stores, including MongoDB, PostgreSQL Couchbase, are schema-free systems that store data in the form of JSON documents. Document stores are similar to key-value or wide column stores, but the document name is the key and the contents of the document, whatever they are, are the value. In a document store, individual records do not require a uniform structure, can contain many different value types, and can be nested. This flexibility makes them particularly well-suited to manage semi-structured data across distributed systems.
4. **Graph databases**, such as Neo4J and Datastax Enterprise Graph, represent data as a network of related nodes or objects in order to facilitate data visualizations and graph analytics. A node or object in a graph database contains free-form data

that is connected by relationships and grouped according to labels. Graph-oriented database management systems (DBMS) software is designed with an emphasis on illustrating connections between data points. As a result, graph databases are typically used when analysis of the relationships between heterogeneous data points is the end goal of the system, such as in fraud prevention, advanced enterprise operations, or Facebook's original friend's graph.

5. **Search engines**, such as elastic search, Splunk, and Solr, store data using schema-free JSON documents. They are similar to document stores, but with a greater emphasis on making your unstructured or semi-structured data easily accessible via text-based searches with strings of varying complexity.

1. Differences between SQL and NoSQL Databases

Having described SQL and NoSQL databases along with their advantages and disadvantages, the table below shows the differences between the two database systems.

Table 4.1: Differences between SQL and NoSQL databases

S/N	SQL	NoSQL
1.	Primary called relational databases	Referred to as non-relational databases
2.	Table based databases	Document based, key-value pairs, graph databases, wide-column stores etc.
3.	Predefined schema for structured data	Dynamic schema for unstructured data
4.	SQL databases are vertically scalable by increasing horse-power of the hardware.	NoSQL databases are horizontally scalable by increasing the database servers in the pool of resources to reduce load.
5.	Structured Query Language (SQL) used for defining and manipulating data.	Unstructured query language is used which varies from database to database.
6.	Emphasizes on ACID properties (Atomicity, Consistency, Isolation and Durability)	Follows the Brewers CAP theorem (Consistency, Availability and Partition Tolerance).

3.8 Conclusion:

Relational databases emerged in the 70s to store data according to a schema that allows data to be displayed as tables with rows and columns. Relational database is a collection of tables, each with a schema that represents the fixed attributes and data types that the items in the table will have. Relational Database Management Systems (RDBMS) provide functionality for reading, creating, updating, and deleting data, typically by means of Structured Query Language (SQL) statements. NoSQL (non-relational) databases emerged as a popular alternative to relational databases as web applications became increasingly complex. NoSQL/non-relational databases can take a variety of forms. However, the critical difference between NoSQL and relational databases is that RDBMS schemas rigidly define how all data inserted into the database must be typed and composed, whereas NoSQL databases can be schema-agnostic, allowing unstructured and semi-structured data to be stored and manipulated.

3.9 References and Further reading

- Agrawal, S., & Patel, A. (2016). A Study ON GRAPH STORAGE DATABASE OF NOSQL. *International Journal on Soft Computing, Artificial Intelligence and Applications (IJSCAI)*, 5(1), 33-39.
- <https://scalegrid.io/blog/2019-database-trends-sql-vs-nosql-top-databases-single-vs-multiple-database-use/>
- <https://www.thegeekstuff.com/2014/01/sql-vs-nosql-db/>
- <https://www.xplenty.com/blog/overview-of-modern-database-systems-which-database-is-right-for-your-use-case/>
- Karthic, S., & Kandasamy, S. A Java Based XML-Driver for Retrieving Data from the XML Document Using SQL-Query.
- Naheman, W., & Wei, J. (2013, December). Review of NoSQL databases and performance testing on HBase. In *Proceedings 2013 International Conference on Mechatronic Sciences, Electric Engineering and Computer (MEC)* (pp. 2304-2309). IEEE.